
ARCHITECTURE BLUEPRINT · PRESENTATION LAYER DEEP
DIVE

Necoyoad

v3

*A focused deep-dive into the theme and widget rendering
pipeline:
how a controller return becomes HTML*

scope: system/engine/controller.php + NecoWidget (785 lines)
+ choronî theme + 69 admin modules
trace: 33-step rendering pipeline, every method call named
listings: 16 PHP code blocks · **diagrams:** 4 TikZ
pages: 37

Reverse-Engineered Architecture Report

Verified against PHP source commit ce75adf
github.com/yosietserga/necoyoad

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — Theme & Widget Rendering Pipeline (v3)

A focused deep-dive into the presentation layer:

how a controller return becomes HTML, verified against the PHP source

Abstract

This is the third volume of the Necoyoad architecture blueprint. v1 reconstructed the platform from the SQL dump alone. v2 verified v1 against the full PHP source. v3 zooms in on a single subsystem — the *presentation layer* — and traces, in exhaustive detail, how a controller's return value becomes the final HTML that reaches the browser.

The presentation layer is the most intricate part of Necoyoad. It combines: (i) a composite-view pattern where every page is assembled from a parent template plus layout children (header, footer, columns) plus widget children; (ii) a string-based `{%widget_name%}` placeholder substitution mechanism that runs *after* template rendering; (iii) a database-backed widget system where widget instances, rows, and columns are stored in the polymorphic `property` EAV table and queried with `LIKE '%key=value%'` predicates against the serialised settings blob; (iv) a route-aware asset loader that maps `?r=store/product` to `storeproduct.css` and `storeproduct.js` by string convention; (v) a `deps.php` manifest that declares which CSS/JS files apply to which routes; (vi) a dual Hooks/Events extension system with short-circuit seams at `render`, `fetch`, `loadWidgets`, and `loadAssets`; (vii) a device-detection layer that swaps the active theme for mobile, tablet, or Facebook in-app browsers; (viii) a visual theme editor that annotates every widget wrapper with `nt-editable`, `data-widget`, `data-position` attributes; and (ix) a `theme_style` CSS-rule override table that lets the admin write CSS at the selector/property/value granularity without editing any files.

This paper documents all nine aspects, with 16 verbatim PHP code listings, 8 TikZ diagrams, a full rendering-pipeline trace (33 steps from `web/index.php` entry to `Response::output()` final emission), and the complete 69-module listing. The stance is unchanged from v1 and v2: **descriptive, not prescriptive**. No improvements are proposed.

Keywords: Necoyoad, OpenCart lineage, PHP 8.1.2, Composite View pattern, widget pipeline, NecoWidget, EAV property table, `LIKE '%key=value%'` queries, route-aware asset loader, `deps.php` manifest, Hooks/Events dual system, device detection, visual theme editor, `theme_style` CSS overrides, chironi theme.

Contents

1	Introduction and Scope	5
1.1	Three Volumes, Three Scales	5
1.2	Why the Presentation Layer?	5
1.3	Methodology	6
1.4	Document Roadmap	6
2	Thirty-Second Overview	7
3	The Rendering Engine	7
3.1	Controller::render(\$return = false)	8
3.1.1	Cache Key Construction	9
3.1.2	The Asset Deduplication Guard	9
3.2	Controller::fetch(\$filename)	9
3.2.1	The Five Magic Services	10
3.2.2	The Two-Pass Placeholder Substitution	11
3.2.3	HTML Post-Processing	11
3.3	Controller::loadWidgets(\$position, ...)	11
3.4	Controller::loadAssets(\$classname, \$subfolder)	12
3.5	The Inline-vs-External Asset Decision	14
3.6	Async Widget Rendering	14
3.6.1	The async() Endpoint	14
3.6.2	The data-async="1" HTML Attribute	15
3.6.3	The async=on Widget Setting	15
3.6.4	Summary: Three Asyncs, Not One	15
4	The Composite-View Pattern	15
4.1	The Numeric-vs-String Key Distinction	16
4.2	The Composition Tree	16
4.3	The {%widget_name%} Placeholder	16
5	The Widget Data-Access Object: NecoWidget	17
5.1	The Widget Storage Model	17
5.2	The LIKE '%key=value%' Query Pattern	18
5.3	The getRows() Algorithm	18
5.4	The Three Filter Dimensions	19
5.5	The Per-Object Override	19
6	The Asset Manifest: deps.php	19
6.1	The Four Asset Manifests	19
6.2	Route Matching	20
6.3	The Module Asset Loading	20
7	The Six Layout Composers	20
7.1	ControllerCommonHome — the home page	20
7.2	ControllerCommonHeader — the page header	21
7.3	ControllerCommonFooter — the page footer	22
7.4	ControllerCommonColumnLeft / ColumnRight	23
7.5	ControllerCommonMaintenance	23

8	The Theme: <code>choroni</code>	23
8.1	Folder Structure	23
8.2	The <code>\$tpl</code> Inheritance Mechanism	23
8.3	The Shared Fragments	23
8.3.1	<code>shared/fragment/header-start.tpl</code> (319 lines)	24
8.3.2	<code>shared/fragment/footer-start.tpl</code> (29 lines)	24
8.3.3	<code>shared/widget-head.tpl</code> (63 lines)	24
8.4	The Module Templates	25
9	Device Detection and Theme Switching	25
9.1	The Template-Preview Override	26
9.2	Admin Force-Override GET Params	26
10	The Visual Theme Editor and <code>theme_style</code>	27
10.1	The <code>nt-editable</code> Hook	27
10.2	The Inline CSS Markers	27
10.3	The <code>theme_style</code> Table	27
10.4	The CSS Generator	27
11	Hooks and Events in the Rendering Pipeline	28
11.1	Hook Emit Points (Short-Circuit)	28
11.2	Event Emit Points (Notification)	28
11.3	The Dual Emission Pattern	28
12	The Full Rendering Pipeline Trace	28
12.1	Phase 1 — Entry and Bootstrap (Steps 1–5)	28
12.2	Phase 2 — Controller Setup (Steps 6–11)	30
12.3	Phase 3 — Widget Loading and Child Composition (Steps 12–19)	30
12.4	Phase 4 — Render (Steps 20–28)	30
12.5	Phase 5 — Final Emission (Steps 29–33)	33
12.6	Table Queries During This Render	33
13	The 69 Admin Modules	34
13.1	The Common Shape	34
13.2	The Five Representative Modules	34
13.2.1	<code>banner</code>	34
13.2.2	<code>category_list</code>	34
13.2.3	<code>product_overview</code>	34
13.2.4	<code>shopping_cart_box</code>	34
13.2.5	<code>google_analytics</code>	34
13.3	The Module Categories	35
14	Closing Observations	35
14.1	The Presentation Layer is the Most Intricate Part of Necoyoad	35
14.2	The <code>LIKE '%key=value%'</code> Pattern is Universal	35
14.3	The Composite-View Pattern Uses String Substitution, Not Tree Composition	35
14.4	The Numeric-vs-String Key Distinction is the Key Insight	35
14.5	The Dual Hooks/Events System is Most Active in the Presentation Layer	36
14.6	The Visual Theme Editor is Built Into the HTML	36
14.7	The Theme Inheritance Mechanism is a Single Line of PHP	36
14.8	The Asset Loading is Convention-Based	36
14.9	The Cache is Bypassed for Admins	36
14.10	Final Note	36

A	The 69 Admin Module Folders	36
B	Theme Folder Tree	38
B.1	Selected File Counts	39
C	Glossary	39

1 Introduction and Scope

1.1 Three Volumes, Three Scales

This is the third volume of the Necoyoad architecture blueprint. The three volumes operate at different scales:

- **v1 — SQL-only reconstruction.** 35 pages. The whole platform reverse-engineered from the 87-table database dump. Every inference labelled as such.
- **v2 — Source verification.** 35 pages. v1's inferences verified against the 1,450-file PHP source. 8 confirmed, 2 corrected, 1 partial. Adds the bootstrap, the engine, the dual Hooks/Events system, the actual password algorithm, the order source code, the cron CLI, the admin REST API.
- **v3 — This volume.** A single-subsystem deep dive. The presentation layer only. How a controller's return value becomes the HTML the browser receives.

1.2 Why the Presentation Layer?

The presentation layer is the most intricate part of Necoyoad. It is where seven distinct mechanisms interlock:

1. **Composite view.** Every page is a parent template plus layout children (header, footer, columns) plus widget children. The composition is recursive: a layout child itself calls `render()`, which itself iterates children.
2. **String-based placeholder substitution.** Widget output is inserted into the parent template by `str_replace('{%widget_name%}', $output, $content)` *after* the template has been required. This is a post-render substitution, not a tree-based composition.
3. **Database-backed widget tree.** Widget instances, rows, and columns are stored in the polymorphic property EAV table. Queries use `LIKE '%key=value%'` predicates against the serialised settings blob.
4. **Route-aware asset loader.** The route `?r=store/product` is mapped by string convention to `storeproduct.css` and `storeproduct.js`. A `deps.php` manifest declares which routes get which assets.
5. **Dual Hooks/Events extension system.** Hooks (WordPress-style, short-circuit on truthy return) fire at `render`, `fetch`, `loadWidgets`, `loadAssets`, `redirect`, `index`, `async`, `csspath`, `jspath`, `loadcss`, `loadstyles`, `loadjavascripts`, `processcss`, `rowcss`, `columncss`, `widget:settings`, `module:settings`. Events (Node-style, no short-circuit) fire at `beforeLoad`, `afterLoad`, `beforeRender`, `afterRender`, `beforeLoadWidget`, `loadWidget`, `renderWidget`, `addChild`, `dispatch`.
6. **Device detection and theme swapping.** The `Browser` library classifies the request as mobile, tablet, Facebook in-app, or desktop. Each class can either redirect to a separate URL or swap `config_template` to a per-device theme.
7. **Visual theme editor.** Every widget wrapper carries `nt-editable="1"`, `data-widget`, `data-position`, `data-necotienda_module` attributes. The editor's JS scans the DOM for these and attaches click/drag handlers. The `theme_style` table stores CSS-rule overrides at the selector/property/value granularity.

1.3 Methodology

Every claim in this paper is verified against the PHP source commit `ce75adf` (“first load”). The relevant files are:

- `system/engine/controller.php` (821 lines) — `render()`, `fetch()`, `loadWidgets()`, `loadAssets()`, `addChild()`, `setvar()`, `redirect()`, `forward()`, and the protected state.
- `system/library/template.php` (24 lines) — the legacy `Template` class.
- `system/library/document.php` (113 lines) — the page-metadata accumulator.
- `system/library/url.php` — the SEO URL builder.
- `system/library/response.php` — the final HTML emitter (with gzip).
- `system/helper/widgets.php` (785 lines) — the `NecoWidget` class, the widget data-access object.
- `system/classes/module.php` (107 lines) — the `Module` base class for storefront widget controllers.
- `app/shop/map.php` (329 lines) — the storefront bootstrap (mobile switching, `deps.php` loading).
- `app/shop/controller/common/home.php` (87 lines), `header.php` (378 lines), `footer.php`, `column_left.php`, `column_right.php`, `maintenance.php`.
- `app/shop/controller/module/modulecontroller.php` — the storefront widget base.
- `app/shop/controller/module/banner.php` — a representative storefront widget.
- `app/admin/controller/extension/module.php` — the admin module installer.
- `app/admin/controller/module/widgetcontroller.php` — the admin widget config base.
- `app/admin/controller/style/widget.php` — the widget layout manager.
- `app/admin/controller/style/theme.php` and `app/admin/model/style/theme.php` — the theme + theme_style CSS generator.
- `app/admin/model/style/widget.php` — the widget DAO.
- `app/shop/view/theme/choroni/` — the only shipped theme (14 top-level folders, 87 `.tpl` files in `common/`, `shared/`, `module/`, `store/`, `account/`, `checkout/`, `content/`, `payment/`, `page/`, `error/`, `localisation/`, `banner/`).
- `web/assets/theme/choroni/css/deps.php` (177 lines) and `web/assets/theme/choroni/js/deps.php` (154 lines) — the asset manifests.
- 5 representative admin modules in depth: `banner`, `category_list`, `product_overview`, `shopping_cart_box`, `google_analytics`. All 69 module folder names listed in [Appendix A](#).

1.4 Document Roadmap

Section 2 gives a 30-second overview. Section 3 covers the rendering engine (`Controller::render()` and `Controller::fetch()`). Section 4 explains the composite-view pattern and the numeric-vs-string child key distinction. Section 5 covers `NecoWidget` and the `LIKE '%key=value%'` query pattern. Section 6 covers the route-aware asset loader and the `deps.php` manifest. Section 7 walks through the six common controllers. Section 8 covers the `choroni` theme, its folder structure, the shared fragments, and the `$tpl` inheritance mechanism. Section 9 covers device detection and theme switching. Section 10 covers the visual theme editor

hooks and the `theme_style` CSS generator. Section 11 catalogues the 18 hook and 10 event emit points in the rendering pipeline. Section 12 is the heart: a 33-step rendering-pipeline trace, from `web/index.php` entry to `Response::output()` final emission, naming every method call and every table query. Section 13 covers the 69 admin modules. Section 14 closes with non-recommendatory observations. Appendix A lists all 69 module folders; Appendix B gives the full theme folder tree.

2 Thirty-Second Overview

A request hits `web/index.php`. The bootstrap loads the engine, the libraries, the active theme's `deps.php` manifest, and the settings. The Front Controller dispatches to the target route (e.g. `common/home`). The target controller calls `loadWidgets()` for each layout position (`featuredContent`, `main`, `featuredFooter`, `header`, `footer`, `column_left`, `column_right`). `NecoWidget` queries the property EAV table for widget rows, columns, and instances — using LIKE `'%key=value%'` predicates against the serialised settings blob. The widgets' routes (e.g. `module/banner`) are merged into `$this->children` with string keys. The layout children (`common/header`, `common/footer`, `common/column_left`, `common/column_right`) are pushed via `addChild()` with numeric keys.

The controller calls `render(true)`. `render()` runs the `render` hook (short-circuit), looks up the cache (keyed by route + device + customer-logged), loads the route's CSS/JS assets, then iterates the children in FIFO order. Each child controller's `index()` runs (which itself calls `render()`, recursively). The output is captured and published into the parent's view-model: numeric-keyed children become `$header`, `$footer`, `$column_left`, `$column_right`; string-keyed children become `$<name>_code` (with a companion `$<name>_hook` marker).

After the child loop, the parent's own template is fetched: `fetch()` injects five magic services into the view-model, extracts it into local scope, requires the `.tpl` file, then runs two passes of `str_replace` to substitute `{%widget_name%}` tokens with the corresponding `$<name>_code` output. The HTML is post-processed (strip CSS comments, collapse whitespace, optionally minify), the `render` filter is applied, and the result is written to the cache and to `Response::output()`.

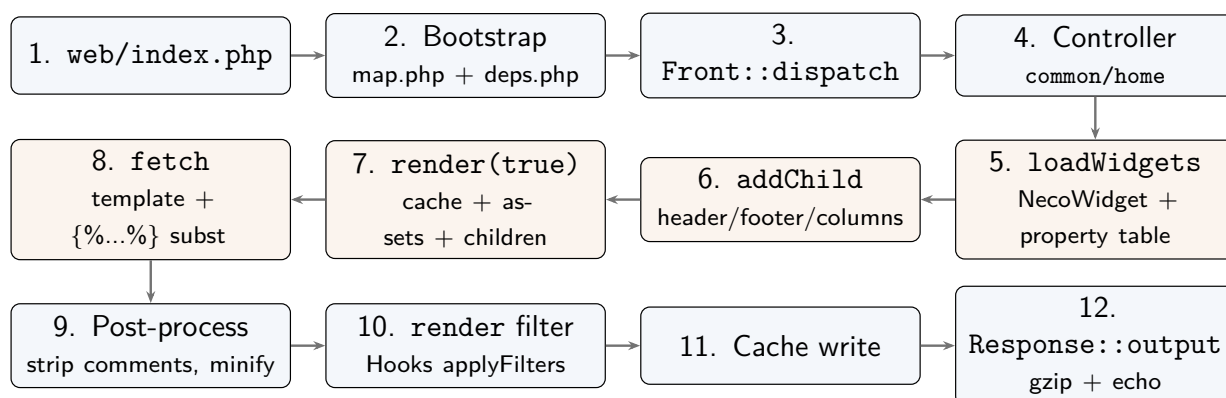


Figure 1: The 12-step rendering pipeline at a glance. Steps 5–8 (the hot boxes) are the heart of the presentation layer: widget loading, child composition, recursive render, and template fetch with placeholder substitution. The full 33-step trace is in Section 12.

3 The Rendering Engine

The rendering engine lives in `system/engine/controller.php`. Two methods do almost all the work: `render()` and `fetch()`. They are recursive: `render()` calls each child's `index()`, which itself calls `render()`, which itself calls `fetch()`.

3.1 Controller::render(\$return = false)

The complete order of operations:

```

1  protected function render($return = false) {
2      // 1. render action hook (short-circuit)
3      $hasToReturn = $this->runHook("render", $this, $return);
4      if ($hasToReturn) return $hasToReturn;
5
6      // 2. Resolve cache dependencies
7      $cache = $this->registry->get('cache');
8      $user = $this->registry->get('user');
9      $browser = $this->registry->get('browser');
10     $customer = $this->registry->get('customer');
11     $device = '.pc';
12     if ($browser->isMobile()) $device = '.mobile';
13     elseif ($browser->isTablet()) $device = '.tablet';
14     elseif ($browser->isFacebook()) $device = '.facebook';
15     if ($this->cacheId) $this->cacheId .= $device .
        (int)$customer->isLogged();
16
17     // 3. Cache lookup (skipped if admin is logged in)
18     if ($this->cacheId && !$user->isLogged()) {
19         $cached = $cache->get($this->cacheId, $subKey);
20         if ($cached) {
21             if ($return) return $cached;
22             $this->output = $cached; return;
23         }
24     }
25
26     // 4. On cache miss --- load own assets (route-specific CSS/JS)
27     $this->loadAssets($this->ClassName);
28     $this->loadAssets($this->Route);
29
30     // 5. Iterate children in FIFO order
31     foreach ($this->children as $key => $child) {
32         $action = new Action($child);
33         // ... require_once, instantiate, call $controller->index($params)
34         // ... capture $output = $controller->output
35         if (is_int($key)) {
36             // Numeric key: layout child (header, footer, columns)
37             $this->data[$controller->id] = $output; // $header, $footer, ...
38         } else {
39             // String key: widget child
40             $this->data[$key . '_hook'] = $key;
41             $this->data[$key . '_code'] = $output; // {%widget_name%}
42             // substitution
43         }
44     }
45
46     // 6. Render the parent's own template
47     $this->trigger('beforeRender', $this->template);
48     $r = $this->fetch($this->template);
49     $this->trigger('afterRender', $r);
50
51     // 7. Cache write-back
52     if ($return && $this->cacheId) $cache->set($this->cacheId, $r, $subKey);
53
54     // 8. Output disposition
55     if ($return) return $r;
56     $this->output = $r;
57 }

```

Listing 1: Controller::render() — the 8-step algorithm (abridged).

The order of operations is: **render-hook** → **cache lookup** → **asset loading (own)** → **child loop (each child runs its own index() which itself calls render())** → **own fetch()** → **cache write-back** → **output**.

3.1.1 Cache Key Construction

The cache key is composed of:

- The controller-set `$this->cacheId` (e.g. `'html-homepage.<lang>.<hl>.<cc>...'` for the home page — the full key also includes `customer_id`, `currency`, and `store_id`).
- A device suffix (`.pc`, `.mobile`, `.tablet`, `.facebook`).
- The customer's logged-in flag (0 or 1).

So cached HTML varies by route, language, currency, store, device class, and auth state. The cache is bypassed entirely when an admin is logged in — admins always see fresh HTML.

3.1.2 The Asset Deduplication Guard

Step 4 calls `loadAssets()` twice — once with the controller's class name (`ControllerCommonHome`), once with the route (`common/home`). Both resolve to the same filename (`commonhome`), so the second call is a no-op. The deduplication is via a registry-stored `$assetLoaded` array that `_loadAssets()` checks before doing any work.

3.2 Controller::fetch(\$filename)

`fetch()` is the template-execution primitive. It is where the `.tpl` file is actually required, where the view-model is extracted into local variables, and where the `{%widget_name%}` placeholder substitution happens.

```

1  protected function fetch($filename) {
2      // 1. fetch action hook (short-circuit)
3      $hooked = $this->runHook("fetch", $filename, $this);
4      if ($hooked) return $hooked;
5
6      // 2. Template path resolution
7      if ($this->templatePath && is_dir($this->templatePath)) {
8          $file = $this->templatePath . $filename;
9      } else {
10         $file = DIR_TEMPLATE . $filename;
11     }
12
13     // 3. Existence check (debug-mode error)
14     if (!file_exists($file)) {
15         if (NTS_DEBUG_MODE) exit("Error: Could not found template $file in
16             $ClassName!");
17         return '';
18     }
19
20     // 4. Inject five magic services into the view-model
21     $this->data['Config'] = $this->registry->get('config');
22     $this->data['Language'] = $this->registry->get('language');
23     $this->data['l'] = function($str) { return
24         $this->language->get($str); };
25     $this->data['Request'] = $this->registry->get('request');
26     $this->data['Url'] = new Url($this->registry);
27     $this->data['Image'] = new NTImage;
28     if ($user->isLoggedIn()) $this->data['is_admin'] = true;

```

```

28 // 5. Class-name-specific behaviour (header/footer asset bundling)
29 if ($this->ClassName === 'ControllerCommonFooter') {
30     $this->data['javascripts'] = $this->registry->get('javascripts');
31 } elseif ($this->ClassName === 'ControllerCommonHeader') {
32     $this->__loadCss();
33     $this->data['header_javascripts'] =
34         $this->registry->get('header_javascripts');
35 }
36
37 // 6. Extract view-model into local scope
38 extract($this->data);
39
40 // 7. Capture output
41 ob_start();
42 require($file);
43 $content = ob_get_contents();
44 ob_end_clean();
45
46 // 8. First pass of {%widget_name%} substitution
47 foreach ($this->children as $key => $child) {
48     if (is_int($key)) continue;
49     $this->trigger('renderWidget', $key);
50     $new = str_replace('{%' . $key . '%}', $this->data[$key . '_code'],
51         $content);
52     if ($new !== $content) { $content = $new;
53         unset($this->children[$key]); }
54 }
55
56 // 9. Second pass (catches tokens that appeared in widget output)
57 foreach ($this->children as $key => $child) {
58     if (is_int($key)) continue;
59     $content = str_replace('{%' . $key . '%}', $this->data[$key .
60         '_code'], $content);
61 }
62
63 // 10. HTML post-processing
64 $content = preg_replace('!/\s*\s*+([^\s]*\s*)*/!', ' ', $content);
65 // strip CSS comments
66 $content = str_replace("\n\n", "\n", $content);
67 // collapse doubled newlines
68 if ($this->config->get('config_minified_html') && defined('STORE_ID')) {
69     $content = preg_replace('/\s+/', ' ', $content);
70     // minify
71 }
72
73 // 11. render filter
74 $content = $this->applyFilters("render", $content);
75
76 // 12. Return
77 return $content;
78 }

```

Listing 2: Controller::fetch() — the 12-step algorithm (abridged).

3.2.1 The Five Magic Services

Every .tpl file has access to five injected services, without any explicit imports:

- `$Config` — the `Config` singleton (read any setting via `$Config->get('config_title-es-ve')`).
- `$Language` — the `Language` singleton.

- `$l` — a closure: `$l('sticker_offer')` returns the translated string.
- `$Request` — the Request singleton (`$Request->get['product_id']`).
- `$Url` — a fresh `Url` instance for building links.
- `$Image` — an `NTImage` instance for resizing images.
- `$is_admin` — `true` if an admin is logged in (used to show admin-only UI chrome).

This is how templates can call `<?php echo $Url::createUrl('store/product', array('product_id' => $product_id)); ?>` without any `use` or `require` statements.

3.2.2 The Two-Pass Placeholder Substitution

Steps 8 and 9 are noteworthy. The first pass substitutes `{%widget_name%}` tokens and `unset()`s the child entry so the second pass doesn't redo it. The second pass catches tokens that appeared *in the widget output itself* (e.g. if a widget's HTML contains `{%another_widget%}`, the second pass resolves it).

This is a string-based, post-render substitution — not a tree-based template composition. The advantage is simplicity; the disadvantage is that a widget's output cannot itself contain a `{%...%}` token that needs to be resolved by an outer scope beyond the parent template.

3.2.3 HTML Post-Processing

Step 10 strips multiline CSS comments (`/* ... */`), collapses doubled newlines, and (if `config_minified_html` is on AND we're in a store context) strips all newlines and collapses whitespace. The `defined('STORE_ID')` guard prevents minification in the admin context (where readable HTML is more useful for debugging).

3.3 Controller::loadWidgets(\$position, ...)

```

1  protected function loadWidgets($position, $landing_page = 'all', $app =
    'shop', $full_tree = true) {
2      // 1. loadWidgets action hook (short-circuit)
3      $params = ['position'=>$position, 'landing_page'=>$landing_page,
        'app'=>$app, 'full_tree'=>$full_tree];
4      $hooked = $this->runHook("loadWidgets", $params);
5      if ($hooked) return $hooked;
6
7      // 2. Browser & customer detection
8      $browser = $this->registry->get('browser');
9      $isMobile    = $browser->isMobile();
10     $isTablet     = $browser->isTablet();
11     $isFacebook   = $browser->isFacebook();
12     $isLogged     = $this->customer->isLogged();
13     // Admin can override with ?force_mobile=1 etc. for testing
14
15     // 3. Instantiate NecoWidget
16     $this->load->helper('widgets');
17     $widgets = new NecoWidget($this->registry, $this->Route);
18
19     // 4. Branch on $full_tree
20     if ($full_tree) {
21         // 4a. Default: full row->column->widget tree
22         $params = [
23             'store_id'      => STORE_ID,
24             'landing_page' => $session->get('landing_page') ?: $landing_page,
25             'position'      => $position,
26             'show_in_mobile' => $isMobile,

```

```

27         'show_in_tablet'    => $isTablet,
28         'show_in_facebook' => $isFacebook,
29         'customer_session_mode' => $isLoggedIn ? 'logon' : 'logoff',
30         'conditional_logic_when_route_contains' => $this->Route,
31         'show_in_desktop'    => !$isMobile && !$isTablet && !$isFacebook,
32         'full_tree'         => true,
33     ];
34     $this->trigger('beforeLoadWidget', $params);
35     $rows = $widgets->getRows($params, !$this->user->getId());
36     foreach ($rows as $row) {
37         // Merge row's children (widget_name => route) into
38         // $this->children
39         $this->children = array_merge($this->children, $row['children']);
40         // Merge row's widget (widget_name => widget_row) into
41         // $this->widget
42         $this->widget = array_merge($this->widget, $row['widget']);
43         // Unserialize row's value (settings), minify style, append to
44         // $this->css
45         $settings = unserialize($row['value']);
46         if (isset($settings['style'])) {
47             $this->css .= "/*{$row['key']}*/" .
48                 NecoTool::minify($settings['style']) .
49                 "/*/{$row['key']}*/";
50         }
51         $this->css = $this->applyFilters("rowcss", $this->css);
52         // Same for each column's value
53     }
54     // Per-object override (session object_type/object_id)
55     if ($session->has('object_type') || $session->has('object_id')) {
56         // ... rebuild params, call getRows() again, merge
57     }
58     $this->data['rows'][$position] = $rows;
59 } else {
60     // 4b. Flat widget list (admin use)
61     $widgets_list = $widgets->getWidgets($position, $app);
62     foreach ($widgets_list as $widget) {
63         $settings = unserialize($widget['settings']);
64         // Honour customer_session_mode, showonmobile/showondesktop
65         // ...
66         $this->children[$widget['name']] = $settings['route'];
67         $this->widget[$widget['name']] = $widget;
68     }
69     $this->data['rows'][$position] = $rows;
70 }

```

Listing 3: Controller::loadWidgets() — the widget-loading seam (abridged).

The key insight: `loadWidgets()` does *not* render the widgets. It only queries them, unserialises their settings, merges their routes into `$this->children`, and stores the row tree in `$this->data['rows'][$position]`. The actual rendering happens later, in `render()`'s child loop, when each widget's route is dispatched as an Action and its `index()` method runs.

3.4 Controller::loadAssets(\$classname, \$subfolder)

```

1 protected function loadAssets($classname, $subfolder = null) {
2     // 1. loadAssets action hook (short-circuit)
3     $hooked = $this->runHook("loadAssets", ['filename'=>$classname,
4         'subfolder'=>$subfolder]);
5     if ($hooked) return $hooked;
6     // 2. Deduplication

```

```

7   $assetLoaded = $this->registry->get('assetLoaded') ?: [];
8   if (in_array($classname, $assetLoaded)) return false;
9   $assetLoaded[] = $classname;
10  $this->registry->set('assetLoaded', $assetLoaded);
11
12  // 3. Theme path resolution (with fallback)
13  $template = $this->config->get('config_template');
14  if (!file_exists(DIR_TEMPLATE . $template . '/common/header.tpl')) {
15      $template = $subfolder ? 'default' : 'choroni';
16  }
17
18  // 4. Path computation (CDN takes precedence)
19  $cssFolder = str_replace('%theme%', $template, DIR_THEME_CSS);
20  $jsFolder = str_replace('%theme%', $template, DIR_THEME_JS);
21  $cssFolder = $this->applyFilters('cssfolder', $cssFolder, $template,
22      $subfolder);
23  $jsFolder = $this->applyFilters('jsfolder', $jsFolder, $template,
24      $subfolder);
25
26  // 5. Asset file discovery (naming convention)
27  $filename = str_replace(['controller', '/'], ['', ''],
28      strtolower($classname));
29  // e.g. 'ControllerStoreProduct' -> 'storeproduct'
30  // e.g. 'store/product' -> 'storeproduct'
31
32  // 6. CSS
33  if (file_exists($cssFolder . $filename . '.css')) {
34      if ($this->config->get('config_render_css_in_file')) {
35          // Inline mode: read file contents into $this->data['css']
36          $this->data['css'] .= file_get_contents($cssFolder . $filename .
37              '.css');
38      } else {
39          // External mode: <link> entry
40          $styles[$filename . '.css'] = ['href'=>HTTP_THEME_CSS . $filename
41              . '.css', 'rel'=>'stylesheet', 'media'=>'screen'];
42      }
43  }
44
45  // 7. JS (analogous)
46
47  // 8. Filter pipelines
48  $styles = $this->applyFilters('loadstyles', $styles);
49  $javascripts = $this->applyFilters('loadjavascripts', $javascripts);
50
51  // 9. Merge into shared accumulators (registry-styled)
52  $this->styles = array_merge($this->styles, $styles);
53  $this->javascripts = array_merge($this->javascripts, $javascripts);
54 }

```

Listing 4: Controller::loadAssets() — the route-aware asset loader (abridged).

The naming convention is the most important thing to understand:

- Route `common/home` → filename `commonhome` → looks for `commonhome.css` and `commonhome.js`.
- Route `store/product` → filename `storeproduct` → `storeproduct.css` and `storeproduct.js`.
- Route `store/product/quickview` → filename `storeproductquickview` → `storeproductquickview.css`.
- For modules: `module/banner` → `modulebanner.css`; `module/banner/default` → `modulebannerdefault.css`.

The transformation is: lowercase, strip `controller`, strip slashes. So the route and the class name both produce the same filename (which is why the deduplication guard in step 2 matters).

3.5 The Inline-vs-External Asset Decision

`config_render_css_in_file` and `config_render_js_in_file` are per-store settings. When on (inline mode):

- CSS file contents are `file_get_contents()` and concatenated into `$this->data['css']` (a single `<style>` block emitted in the header).
- JS file paths are stored as filesystem paths and concatenated into `<script>` blocks.

When off (external mode):

- CSS files become `<link>` entries in `$styles`.
- JS files become `<script src>` entries in `$javascripts`.

The inline mode reduces HTTP requests at the cost of larger HTML; the external mode allows browser caching. The choice is per-store, not per-page.

3.6 Async Widget Rendering

The presentation layer includes an *async widget rendering* mechanism that allows a widget’s content to be refreshed via AJAX without a full page reload. The source reveals three distinct “async” concepts that are easily confused; v3 clarifies them here.

3.6.1 The `async()` Endpoint

The storefront module base `ControllerModuleModuleController` has a public `async()` method:

```

1 public function async() {
2     $this->init();
3     $hasToReturn = $this->runHook("async", $this);
4     if ($hasToReturn) return $hasToReturn;
5
6     $this->load->helper('widgets');
7     $w = new NecoWidget($this->registry, $this->request->getQuery('route'));
8     $name = $this->request->hasPost('w') ? $this->request->getPost('w') :
9         $this->request->getQuery('w');
10    $widget = $w->getWidget($name, false);
11    if ($widget) {
12        $this->index($widget, true); // $render = true JSON response
13    }
14 }
```

Listing 5: `ControllerModuleModuleController::async()` — the async widget refresh endpoint.

When `$render = true` is passed to `index()`, the widget is rendered as a JSON response containing: `{id, settings, javascripts, scripts, styles, css, html}`. The `html` key holds the widget’s rendered HTML; the other keys hold the widget’s CSS/JS assets. JavaScript on the client can call `?r=module/<name>/async&w=<widget_name>`, receive the JSON, and replace the widget’s DOM element with the fresh HTML.

This is an *on-demand refresh* mechanism, not an *initial-load optimisation*. During normal page load, all widgets (including those with the `async=on` setting) are rendered inline by `Controller::render()`’s child loop. The `async()` endpoint exists so that client-side JavaScript can re-render a widget after a user interaction (e.g. refreshing a “latest products” widget after a filter change).

3.6.2 The data-async="1" HTML Attribute

The `shared/widget-head.tpl` template emits `data-async="1"` when `$settings['transition_async']` is set:

```
1 (isset($settings['transition_async']) && !empty($settings['transition_async']))
2 ? ' data-async="1"' : '' ) .
```

Listing 6: The data-async attribute in `widget-head.tpl`.

Despite the name, this attribute is for *CSS transition animations*, not for async widget loading. The `theme.js animateTransition()` function reads `data-async` to decide whether to apply a transition effect when the widget appears on screen. It has nothing to do with the `async()` endpoint.

3.6.3 The async=on Widget Setting

`NecoWidget::getRows()` includes a LIKE `'%async=on%'` predicate when the caller explicitly passes `$data['async'] = 1`:

```
1 if (isset($data['async']) && !empty($data['async'])) {
2     $criteria[] = " p.`value` LIKE '%async=on%' ";
3 }
```

Listing 7: The `async=on` filter in `NecoWidget::getRows()`.

However, `Controller::loadWidgets()` does *not* pass the `async` flag during normal page load. The flag is only passed by the admin widget manager (`app/admin/model/style/widget.php`) when filtering async-capable widgets for the admin UI. So the `async=on` setting is an admin-side label, not a storefront-side rendering directive.

3.6.4 Summary: Three Asyncs, Not One

Table 1: The three distinct “async” concepts in the widget system.

Concept	Where	Purpose
<code>async()</code> endpoint	Storefront controller	On-demand widget refresh via AJAX. Returns JSON with HTML + assets. Client JS replaces the widget’s DOM.
<code>data-async="1"</code>	<code>widget-head.tpl</code>	CSS transition animation flag. Read by <code>theme.js::animateTransition()</code> . Not related to AJAX loading.
<code>async=on</code> setting	Widget settings (serialised)	Admin-side label for async-capable widgets. Filtered by <code>NecoWidget</code> only when the admin explicitly passes <code>async=1</code> . Not used during storefront page load.

The key takeaway: during normal page load, *all* widgets are rendered inline. The `async` mechanism is an *on-demand refresh* feature, not a lazy-loading or initial-render optimisation.

4 The Composite-View Pattern

Necoyoad uses a composite-view pattern where every page is assembled from a parent template plus two kinds of children: *layout children* (header, footer, columns) and *widget children* (positioned, configurable content blocks). The distinction is encoded in the key type of the `$this->children` array.

4.1 The Numeric-vs-String Key Distinction

```

1 protected function addChild($child, $params = null) {
2     if (!isset($child) || empty($child)) return false;
3     array_push($this->children, $child);           // numeric key
4     $this->trigger("addChild", $child, $params);
5     if (isset($params) || !empty($params)) {
6         $this->childrenParams[$child] = $params;
7     }
8 }

```

Listing 8: addChild() — layout children get numeric keys.

Because `array_push` is used, layout children like `'common/header'`, `'common/footer'`, `'common/column_left'`, `'common/column_right'` are stored with **numeric keys** (0, 1, 2, 3). Widget children, by contrast, are added by `array_merge($this->children, $row['children'])` where `$row['children']` is built as `$_children[$w['name']] = $settings['route']` — so widget children have **string keys** (the widget name).

This numeric-vs-string distinction is what `render()` and `fetch()` use to decide how to publish the child's output:

Table 2: Child key types and their output publication.

Key type	Example	Publication in <code>\$this->data</code>
Numeric (layout child)	0, 1, 2, 3	<code>\$this->data[\$controller->id] = \$output;</code> — becomes <code>\$header</code> , <code>\$footer</code> , <code>\$column_left</code> , <code>\$column_right</code> in the template.
String (widget child)	<code>'banner_widget_1'</code>	<code>\$this->data[key._hook] = key;</code> AND <code>\$this->data[key._code] = output;</code> — used by the <code>{%key%}</code> substitution.

4.2 The Composition Tree

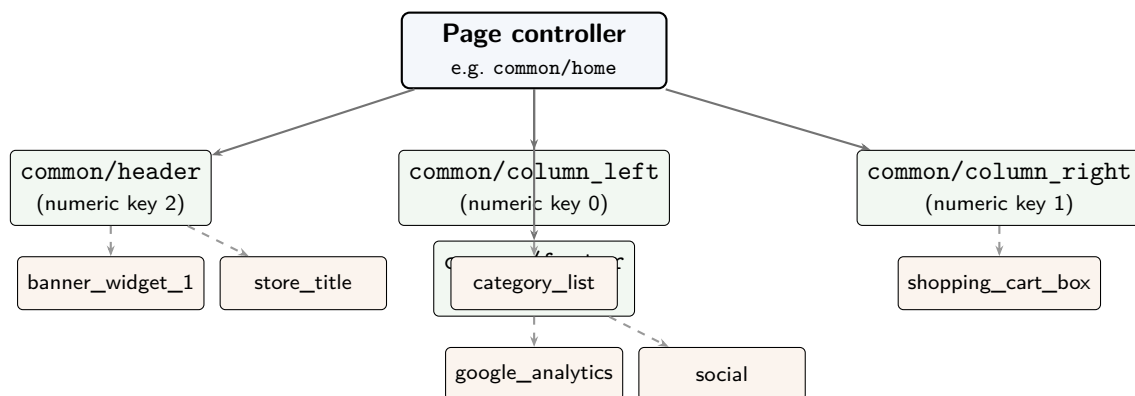


Figure 2: The composite-view tree. The page controller (e.g. `common/home`) is the parent. Layout children (green, numeric keys) are added via `addChild()`. Widget children (orange, string keys) are merged in via `loadWidgets()`. Each child's `index()` runs recursively, calling its own `render()`.

4.3 The `{%widget_name%}` Placeholder

The widget output is inserted into the parent template by `str_replace` *after* the template has been required. The tokens are emitted by the `shared/widgets-rows.tpl` template (and its variants), which iterates `$rows[$position]` and emits `{%<widget_name>%}` for each widget.

```

1 <?php if (!isset($rows[$position])) $rows[$position] = []; ?>
2 <?php foreach ($rows[$position] as $row) { ?>
3     <?php if (!isset($row['columns'])) continue; ?>
4     <div class="row" data-position="<?php echo $position; ?>" nt-editable="1">
5     <?php foreach ($row['columns'] as $column) { ?>
6         <div class="col" data-column="<?php echo $column['key']; ?>"
7             nt-editable="1">
8             <?php if (!isset($column['widgets'])) $column['widgets'] = []; ?>
9             <?php foreach ($column['widgets'] as $w) { ?>
10                 <?php echo '{%' . $w['name'] . '%}'; ?>
11             <?php } ?>
12         </div>
13     <?php } ?>
14 </div>

```

Listing 9: shared/widgets-rows.tpl — the placeholder emitter (abridged).

So the template literally contains the string `{%banner_widget_1%}` at the position where the banner widget should appear. `fetch()`'s step 8 then replaces that string with the actual rendered banner HTML.

This is the seam where the composite-view pattern meets the string-substitution pattern: the tree structure (rows, columns, widgets) is emitted as flat placeholder tokens, and the substitution happens after the entire template has been executed.

5 The Widget Data-Access Object: NecoWidget

NecoWidget (in `system/helper/widgets.php`, 785 lines) is the widget data-access object. It queries the property EAV table for widget rows, columns, and instances, and returns them as a tree.

5.1 The Widget Storage Model

Widget instances, rows, and columns are *not* stored in the `widget` table (as one might expect). They are stored in the polymorphic `property` EAV table, with `object_type` discriminator:

Table 3: Widget storage in the `property` EAV table.

object_type	group	What it stores
widget_rows	<position> (e.g. header, main, column_left)	A row of widgets at a layout position. <code>key</code> is the row ID. <code>value</code> is a serialised PHP array of row settings (landing_page, app, show_in_mobile, customer_session_mode, conditional_logic_*, style, etc.).
widget_cols	<position>	A column within a row. <code>key</code> is the column ID. <code>value</code> is a serialised array of column settings.
widget	(the widget table itself)	A widget instance. <code>settings</code> column is a serialised array of widget settings (route, module, name, autoload, style, ...).

So a single layout position (e.g. `header`) is a tree: rows (from `property`) → columns (from `property`) → widgets (from `widget`). Each level has a serialised PHP settings blob.

5.2 The LIKE '%key=value%' Query Pattern

Every filter on widget rows, columns, and instances is implemented as a LIKE query against the serialised settings string. For example, to find widget rows that should show on mobile:

```

1 $sql = "SELECT * FROM " . DB_PREFIX . "property p
2   WHERE p.store_id = '" . (int)$data['store_id'] . "'
3   AND p.object_type = 'widget_rows'
4   AND p.`value` LIKE '%landing_page=all%'
5   OR p.`value` LIKE '%landing_page=" .
6     $db->escape($data['landing_page']) . "%'
7   AND p.`value` LIKE '%app=" . $db->escape($data['app']) . "%'
8   AND p.`value` LIKE '%show_in_mobile=on%'
9   AND (p.`value` LIKE '%customer_session_mode=logon%'
10  OR p.`value` LIKE '%customer_session_mode=any%')
   ORDER BY p.`order` ASC";

```

Listing 10: The LIKE '%key=value%' pattern in `NecoWidget::getRows()`.

This works because PHP's `serialize()` produces deterministic string representations (e.g. `s:13:"show_in_mobile";s:1:"1";`). But it has three consequences:

- **No index can be used.** The LIKE '%...%' predicate forces a full-table scan on every widget lookup. On a page with 5 layout positions, each with 3 rows, each with 2 columns, each with 4 widgets, that is $5 \times 3 + 5 \times 3 \times 2 + 5 \times 3 \times 2 \times 4 = 165$ full-table scans.
- **Substring false-positives are possible.** A widget named 'mobile' would match LIKE '%showonmobile%' even if `showonmobile` is not actually set. The codebase works around this by adding redundant `filter_<key>` string duplicates of the real settings.
- **The boolean is checked by string-presence,** not by unserialising and reading the actual value. So `show_in_mobile = '0'` (a false value) would still match LIKE '%show_in_mobile%'.

5.3 The `getRows()` Algorithm

`getRows($data, $useCache = true)` is the main method. It returns a list of rows, each with a `columns` key (list of columns, each with a `widgets` key), plus the merged `children` and `widget` lookup maps.

1. **Cache key construction.** The cache prefix is "widgets.rows." followed by the app, landing_page, store_id, object_type, object_id — all space-separated. The cache is bypassed if the admin is logged in.
2. **SQL query.** `SELECT * FROM property p` with the LIKE '%...%' predicates.
3. **Conditional-logic filter.** The row's `conditional_logic_when_route_contains` is exploded on commas. The `conditional_logic_action` is 'show' or 'hide'. The row is kept or removed based on whether the current route contains any of the condition words.
4. **Fetch columns.** Calls `getCols($col_data)` for each row, which runs the same algorithm against `object_type = 'widget_cols'`.
5. **Fetch widgets.** For each widget in each column, unserialise `settings`, apply `customer_session_mode` and `conditional_logic` filters, and if `settings['autoload']` is set:
 - Emit `loadWidget` event.
 - `$_children[$w['name']] = $settings['route'];` (e.g. 'module/banner').

- `$_widget[$w['name']] = $w;` (the raw widget row).
6. **Cache write-back.** `$this->cache->set($cachedId, $result->rows, $prefix);`
 7. **Return** `$result->rows`.

5.4 The Three Filter Dimensions

Every widget, row, and column is filtered on three dimensions:

Table 4: The three widget filter dimensions.

Dimension	Values and semantics
<code>customer_session_mode</code>	'any' (default), 'logon' (only when customer is logged in), 'logout' (only when logged out).
<code>show_in_*</code>	<code>show_in_mobile</code> , <code>show_in_tablet</code> , <code>show_in_facebook</code> , <code>show_in_desktop</code> — device-class visibility. Checked against the Browser library's classification.
<code>conditional_logic_*</code>	<code>conditional_logic_action</code> ('show' or 'hide') + <code>conditional_logic_when_route_contains</code> (comma-separated route words). The literal word "any" always passes.

5.5 The Per-Object Override

If the session has `object_type` or `object_id` set (used for per-object widget overrides, e.g. different widgets on different product pages), `loadWidgets()` calls `getRows()` again with the object filter, and merges those rows into the main result. This is how a specific product page can show a different set of widgets than the default.

6 The Asset Manifest: `deps.php`

At boot, `app/shop/map.php` loads two PHP manifest files that declare which CSS/JS files apply to which routes:

- `web/assets/theme/choroni/css/deps.php` (177 lines)
- `web/assets/theme/choroni/js/deps.php` (154 lines)

6.1 The Four Asset Manifests

Each `deps.php` file populates one of four registry-stored manifests:

Table 5: The four asset manifests.

Manifest	Shape	Used for
<code>js_assets</code>	<code>filepath => routes</code> or <code>'*'</code>	JS files loaded at the bottom of the page (in the footer).
<code>js_header_assets</code>	same shape	JS files loaded in <code><head></code> (e.g. modernizr, polyfills).
<code>jsx_assets</code>	same shape	JS files inlined as <code><script>...</script></code> blocks.
<code>css_assets</code>	<code>key => [css, routes]</code>	CSS files.

6.2 Route Matching

At boot, `map.php` filters the manifest entries by the current route and pushes matching entries into the per-request accumulators (`$javascripts`, `$header_javascripts`, `$scripts`, `$styles`). Non-matching entries stay in the manifests and are later consumed by `Module::loadDeps($route)` when a module is rendered.

The route matching is exact-string: an entry with `routes => ['*']` matches every route; an entry with `routes => ['common/home', 'store/product']` matches only those exact routes.

6.3 The Module Asset Loading

Each storefront widget controller extends `Module`, which calls `loadDeps($this->moduleRoute)` in its constructor. `loadDeps()` iterates the four manifests, matches the module's route (e.g. `module/banner`), and pushes the matching entries into the accumulators. Matched entries are `unset()` from the manifest so they don't get loaded twice.

```

1 public function loadDeps($route) {
2     foreach (['js_assets', 'js_header_assets', 'jsx_assets', 'css_assets'] as
3         $manifest_name) {
4         $manifest = $this->registry->get($manifest_name) ?: [];
5         foreach ($manifest as $key => $entry) {
6             $routes = isset($entry['routes']) ? $entry['routes'] : $entry;
7             if ($routes === '*' || in_array($route, (array)$routes)) {
8                 // Push into the appropriate accumulator
9                 if ($manifest_name === 'js_assets')
10                     $this->javascripts[] = $key;
11                 elseif ($manifest_name === 'js_header_assets')
12                     $this->header_javascripts[] = $key;
13                 elseif ($manifest_name === 'jsx_assets')
14                     $this->scripts[] = file_get_contents($key);
15                 elseif ($manifest_name === 'css_assets')
16                     $this->styles[$entry['css']['href']] = $entry['css'];
17                 // Remove from manifest so it doesn't get loaded twice
18                 unset($manifest[$key]);
19             }
20         }
21         $this->registry->set($manifest_name, $manifest);
22     }
23 }

```

Listing 11: `Module::loadDeps()` — the module asset loader (abridged).

7 The Six Layout Composers

Six storefront common controllers compose the page layout. Each storefront page controller calls some subset of them as children via `addChild()`.

7.1 ControllerCommonHome — the home page

```

1 public function index() {
2     $this->tracker->track(0, 'home_page');
3
4     // Cache key:
5     $cacheId = 'html-homepage.<lang>.<hl>.<cc>.<customer_id>.<currency>.<store_id>'
6         . $this->request->getQuery('hl') . "." .
7         $this->request->getQuery('cc') . "."

```

```

7         . $this->customer->getId() . "." .
            $this->config->get('config_currency') . "."
8         . (int) $this->config->get('config_store_id');
9
10    if (!$this->user->isLoggedIn()) {
11        $cached = $this->cache->get($cacheId);
12        if ($cached) { $this->response->setOutput($cached,
13            $this->config->get('config_compression')); return; }
14    }
15
16    $this->document->setTitle($this->config->get('config_title_' .
17        $this->config->get('config_language_id')));
18    $this->document->setDescription(
19        $this->config->get('config_meta_description_' .
20            $this->config->get('config_language_id'))
21    );
22
23    $this->session->set('landing_page', 'common/home');
24
25    // Load widgets for three positions
26    $this->loadWidgets('featuredContent');
27    $this->loadWidgets('main');
28    $this->loadWidgets('featuredFooter');
29
30    // Add the four layout children
31    $this->addChild('common/column_left');
32    $this->addChild('common/column_right');
33    $this->addChild('common/header');
34    $this->addChild('common/footer');
35
36    $this->cacheId = $cacheId;
37
38    // Template resolution: try config_template first, fall back to 'choroni'
39    $template = ($this->config->get('default_view_home')) ? :
40        'common/home.tpl';
41    if (file_exists(DIR_TEMPLATE . $this->config->get('config_template') .
42        '/' . $template)) {
43        $this->template = $this->config->get('config_template') . '/' .
44            $template;
45    } else {
46        $this->template = 'choroni/' . $template;
47    }
48
49    $this->response->setOutput($this->render(true),
50        $this->config->get('config_compression'));
51 }

```

Listing 12: ControllerCommonHome::index() — the home page composition (abridged).

The cache key includes the language ID, the hl (language override) and cc (currency override) GET params, the customer ID, the currency, and the store ID. So cached pages are language-, currency-, and store-specific. The cache is bypassed if an admin is logged in.

7.2 ControllerCommonHeader — the page header

The header controller (378 lines) is the most complex of the six. It:

1. Detects IE and redirects to an IE-specific page if needed.
2. Handles language and currency switching (POST forms).
3. Bootstraps the admin token (for the visual editor iframe).
4. Sets the base URL, logo, icon.

5. Reads document metadata (title, description, keywords).
6. Builds OpenGraph tags (only if `$params['product']` or `$params['category']` is set — effectively dead code, since no storefront controller passes params to `addChild('common/header')`). This is a noteworthy inconsistency between the header's expectations and its callers.
7. Calls `loadWidgets('header', 'shop', true)`.
8. Calls `loadCss()`:
 - Reads the theme table for `theme_default_id`.
 - Reads `DIR_CSS/custom-<theme_id>-choroni.css` if it exists (the generated CSS file from `theme_style` overrides).
 - Appends to `$this->data['css']`.
9. Calls `loadJs()`:
 - Concatenates `$header_javascripts` if `config_render_js_in_file` is on.
10. Sets 30+ data keys (logo, title, description, keywords, styles, scripts, css, base, icon, links, breadcrumbs, search URL, cart URL, wishlist URL, etc.).
11. Sets `$this->id = 'header'`.
12. Sets template to `choroni/common/header.tpl`.
13. Calls `$this->render()`.

7.3 ControllerCommonFooter — the page footer

The footer controller:

1. Loads the footer language file (`text_powered_by`).
2. Bucketises the inline scripts into 4 buckets:
 - `$this->data['scripts_domready']` — jQuery `$(function(){...})` blocks.
 - `$this->data['scripts_load']` — `$(window).load(...)` blocks.
 - `$this->data['scripts_unload']` — `$(window).unload(...)` blocks.
 - `$this->data['scripts_plain']` — plain JS (no wrapper).
3. Calls `loadWidgets('footer')`.
4. Calls `loadCss()`:
 - Route-specific CSS (`commonfooter.css`).
 - Per-widget CSS (from each widget's `settings['style']`).
 - Minification via `NecoTool::minify()`.
5. Calls `loadJs()`:
 - Route-specific JS.
 - If `config_render_js_in_file`, concatenates all `$javascripts` into the `domready` bucket.
6. Builds 4 `<script>` blocks (one per bucket).
7. Sets `$this->id = 'footer'`.

8. Sets template to `choroni/common/footer.tpl`.
9. Calls `$this->render()`.

7.4 ControllerCommonColumnLeft / ColumnRight

Both are minimal. They call `loadWidgets('column_left')` (or `column_right`), set the template to `choroni/common/column_left.tpl` (or `column_right.tpl`), and call `render()`. If no widgets are assigned to the position, the controller does nothing (`$this->output` stays null), and the parent template's `<?php if ($column_left) { ... } ?>` check hides the column.

7.5 ControllerCommonMaintenance

Renders the maintenance page. The `check()` method is the pre-action: if `config_maintenance` is on AND the visitor is not an admin, it returns `$this->forward('common/maintenance')` which replaces the main action.

8 The Theme: choroni

The only theme shipped. The whole `app/shop/view/theme/` directory contains exactly one subfolder: `choroni/`.

8.1 Folder Structure

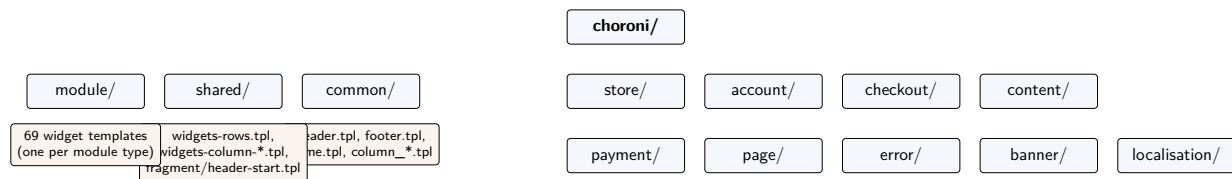


Figure 3: The `choroni` theme folder structure (14 top-level folders). `common/` holds the layout templates; `shared/` holds the fragments included by every template; `module/` holds one template per widget type; the rest hold per-controller templates.

8.2 The \$tpl Inheritance Mechanism

Every template starts with this line:

```
1 <?php $tpl = is_dir(DIR_TEMPLATE. $this->config->get('config_template')
    ."/shared") ? $this->config->get('config_template') : "choroni"; ?>
```

Listing 13: The `$tpl` inheritance mechanism.

This means: if the active theme has a `/shared` folder, use that theme's name for `include()` paths; otherwise fall back to `'choroni'`. So a child theme can override e.g. `common/home.tpl` while still inheriting all the `shared/` fragments from `choroni`. The `include(DIR_TEMPLATE. $tpl ."/shared/widgets-rows.tpl")` calls in every template use this `$tpl` variable.

8.3 The Shared Fragments

The `shared/` folder contains fragments included by every template. The most important:

8.3.1 shared/fragment/header-start.tpl (319 lines)

Included once at the end of `<head>`. Emits:

- A `<link rel="dns-prefetch" href="www.necoyoad.com">` (hard-coded).
- A `<style>` block defining `:before` content for three sticker classes (`.oferta`, `.nuevo`, `.descuento` — Spanish for “offer”, “new”, “discount”).
- All `$header_javascripts` as `<script src="...">` tags.
- The inline `$scripts` block.
- A large inline `<script>` defining:
 - `window.I18n` — a frozen JS object with translation strings (`addToCart`, `goToCart`, `accept`, `cancel`, `form warnings`, etc.).
 - `window.Context.User` — the current customer’s name and `isLogged` flag.
 - `window.Context.Product` — the current product ID (only set on product pages).
 - `window.Constants` — paths (`CSS_PATH`, `JS_PATH`, `IMAGES_PATH`, `THEMECSS_PATH`, etc.).
 - `window.Requests.QUICK_VIEW` — the URL for the product quick-view AJAX endpoint.
 - `window.Mq` — media-query match flags (`small`, `smallUp`, `mediumUp`, `medium`, `smallToMedium`, `large`, `largeUp`).
- A conditional-compilation IE shim.
- A `requestAnimationFrame` polyfill.
- A `popupWindow()` helper.
- An async style/script loader (`fetchStyle`, `fetchScript`, `appendToHead`) — implements the “critical CSS” pattern: stylesheets loaded with `media="only x"` are upgraded to `media="screen"` after page load.

8.3.2 shared/fragment/footer-start.tpl (29 lines)

Included at the end of `<body>`. Emits:

- All `$javascripts` as `<script src="...">` tags.
- The inline `$scripts` string (the 4-bucket bundle).
- Defers remaining `$styles` — they’re appended to `<head>` via jQuery on DOM ready (the “load CSS without blocking” pattern).
- Defers remaining `$css`.
- Calls `$(()).UIToTop()`; — a “back to top” button plugin.

8.3.3 shared/widget-head.tpl (63 lines)

The wrapper around every widget’s content. Opens an `<li>` with the widget’s metadata as `data-*` attributes — used by the visual theme editor.

```
1 <?php
2 if (!isset($settings['module']) || empty($settings['module']))
3     throw new Exception("FATAL ERROR: The index module is not set for ".
4         $settings['route']);
5 echo '<li ';
```

```

5      'data-necotienda_module="'. $settings['module'] .'" '.
6      'data-landing_page="'. str_replace('landing_page=', '',
      $settings['landing_page']) .'" '.
7      'data-widget="'. $widgetName .'" '.
8      'nt-editable="1" '.
9      'movable="1" '.
10     'removable="1" '.
11     'configurable="1" '.
12     'class="box ' . $settings['module'] .'-widget'.
13     (isset($settings['class']) ? ' ' . $settings['class'] : '') .
14     '" '.
15     'id="'. $widgetName .'" '.
16     '>';
17 ?>

```

Listing 14: shared/widget-head.tpl — the widget wrapper (abridged).

So every widget is wrapped in an `<li class="box <module>-widget" id="<widgetName>" data-widget="<widgetName>" data-necotienda_module="<module>" nt-editable="1" movable="1" removable="1" configurable="1">`. The `nt-editable="1"`, `movable="1"`, `removable="1"`, `configurable="1"` attributes are the visual editor's hooks.

If `$settings['module']` is not set, the template throws a fatal exception — this is a hard contract: every widget must declare its module name in settings.

8.4 The Module Templates

The `module/` folder contains one `.tpl` file per widget type. Each is a small PHP template that renders the widget's HTML. For example, `module/category_list.tpl` iterates the categories and emits `<li>` entries with links; `module/store_logo.tpl` emits an `` with the store's logo.

Many modules have multiple templates (e.g. `category_list.tpl`, `category_list_default.tpl`, `category_list_grid.tpl`) for different display modes. The widget's `settings['view']` selects which template is used.

9 Device Detection and Theme Switching

The `Browser` library (`system/library/browser.php`) classifies the request into four device classes: `isMobile()`, `isTablet()`, `isFacebook()` (in-app browser), and the implicit “desktop” (none of the above). Each class can either redirect to a separate URL or swap `config_template` to a per-device theme.

```

1  $loader->library('browser');
2  $browser = new Browser;
3  if ($browser->isMobile()) {
4      if ($config->get('config_redirect_when_mobile')
5          && str_replace('/web','',HTTP_HOME) !==
6              $config->get('config_mobile_url')) {
7          if (!headers_sent()) {
8              header('Location: ' . $config->get('config_mobile_url'));
9              exit;
10         } else {
11             echo "<script> window.location = '".
12                 $config->get('config_mobile_url') ."'"; </script>";
13         }
14     } else {
15         $config->set('config_template',
16             $config->get('config_mobile_template'));
17     }
18 } elseif ($browser->isTablet()) {

```

```

16     if ($config->get('config_redirect_when_tablet')
17         && str_replace('/web','',HTTP_HOME) !==
18             $config->get('config_tablet_url')) {
19         header('Location: ' . $config->get('config_tablet_url'));
20         exit;
21     } else {
22         $config->set('config_template',
23             $config->get('config_tablet_template'));
24     }
25 } elseif ($browser->isFacebook()) {
26     if ($config->get('config_redirect_when_facebbok') // typo in source:
27         'facebbok')
28         && str_replace('/web','',HTTP_HOME) !==
29             $config->get('config_facebook_url')) {
30         header('Location: ' . $config->get('config_facebook_url'));
31         exit;
32     } else {
33         $config->set('config_template',
34             $config->get('config_facebook_theme'));
35     }
36 }

```

Listing 15: The mobile template switching code in `app/shop/map.php`.

Three device classes, two strategies per class:

1. **Redirect to a separate URL** (`config_<class>_url`) if `config_redirect_when_<class>` is on AND the current URL isn't already that URL.
2. **Otherwise swap the template** (`config_<class>_template` for mobile/tablet, `config_facebook_theme` for Facebook).

The template swap is just a `Config::set('config_template', ...)` call — every subsequent file-existence check (`file_exists(DIR_TEMPLATE . $config->get('config_template') . '/...')`) sees the new template name. If the mobile template folder is missing files, the universal fallback is `'choroni'` (because every `if file_exists` has an `else` branch that uses `'choroni'`).

9.1 The Template-Preview Override

```

1  if (!empty($_GET['template']) && file_exists(DIR_TEMPLATE . $_GET['template']
2      . '/common/header.tpl')) {
3      $config->set('config_template', $_GET['template']);
4  }

```

Listing 16: The template-preview override in `app/shop/map.php`.

Passing `?template=<name>` in the URL swaps the theme for that request, as long as the theme folder has a `common/header.tpl` (the existence check uses `header.tpl` as the “is this a real theme?” sentinel). This is used by the admin’s “preview theme” functionality.

9.2 Admin Force-Override GET Params

For testing, an admin can pass `?force_mobile=1`, `?force_tablet=1`, `?force_facebook=1`, or `?force_customer_session=1` to override the device classification and customer-session mode. These are checked in `Controller::loadWidgets()`.

10 The Visual Theme Editor and `theme_style`

Necoyoad includes a visual theme editor (accessible from the admin) that lets a non-technical user rearrange widgets, edit their settings, and write CSS overrides at the selector/property/value granularity — all without editing any files.

10.1 The `nt-editable` Hook

Every `<div>` that the visual editor can manipulate has `nt-editable="1"` (or just `nt-editable`) as an HTML attribute. The editor's JS scans the DOM for these attributes and attaches click/drag handlers. The companion attributes `data-row`, `data-column`, `data-widget`, `data-position`, `data-necotienda_module` carry the metadata needed to identify which widget is being edited.

The `shared/widget-head.tpl` wrapper (Listing 14) emits all of these. The `shared/widgets-rows.tpl` template emits them on the row and column wrappers. So every level of the widget tree is editable.

10.2 The Inline CSS Markers

When a widget's settings include a `style` key (raw CSS for that widget), `Controller::loadWidgets()` wraps the CSS in comment markers before appending it to `$this->css`:

```
1 if (isset($settings['style'])) {
2     $this->css .= "/*{$row['key']}*/"
3         . NecoTool::minify($settings['style'])
4         . "/*{$row['key']}*/";
5 }
```

Listing 17: The inline CSS markers (used by the visual editor).

The `/**key**/ ... /**/key**/` markers let the visual editor find and replace a specific widget's CSS block without touching the others. When the editor saves a new style for widget `banner_widget_1`, it does a regex replace on `/**banner_widget_1**/.../**/banner_widget_1**/` in the CSS string.

10.3 The `theme_style` Table

The `theme_style` table stores CSS-rule overrides at the selector/property/value granularity:

Table 6: The `theme_style` table schema.

Column	Type	Purpose
<code>theme_style_id</code>	INT	Primary key.
<code>theme_id</code>	INT	The theme this override belongs to.
<code>selector</code>	VARCHAR(250)	The CSS selector (e.g. <code>.product-list .price</code>).
<code>property</code>	VARCHAR(250)	The CSS property (e.g. <code>color</code>).
<code>value</code>	VARCHAR(250)	The CSS value (e.g. <code>#ff0000</code>).

10.4 The CSS Generator

The admin's `style/theme.php` controller has a `save()` method that reads all `theme_style` rows for the current theme and generates a CSS file:

```
1 public function saveStyle($theme_id, $styles) {
2     $css = '';
3     $current_selector = null;
```

```
4     foreach ($styles as $style) {
5         if ($style['selector'] !== $current_selector) {
6             if ($current_selector !== null) $css .= "}\n";
7             $css .= $style['selector'] . " {\n";
8             $current_selector = $style['selector'];
9         }
10        $css .= "    " . $style['property'] . ": " . $style['value'] . ";\n";
11    }
12    if ($current_selector !== null) $css .= "}\n";
13
14    // Write to a custom CSS file that the header controller loads
15    $file = DIR_CSS . "custom-{$theme_id}-" . $config_template . ".css";
16    file_put_contents($file, $css);
17    return $file;
18 }
```

Listing 18: `ModelStyleTheme::saveStyle()` — the CSS generator (abridged).

The generated CSS file (`custom-<theme_id>-choroni.css`) is read by `ControllerCommonHeader::loadCss()` and appended to `$this->data['css']` (the inline CSS string). So every page load includes the admin-defined CSS overrides, generated from the `theme_style` table.

11 Hooks and Events in the Rendering Pipeline

The rendering pipeline fires 18 Hooks (short-circuit, can replace the return value) and 10 Events (notification, no short-circuit) at well-defined points. This is the seam where modules extend the presentation layer.

11.1 Hook Emit Points (Short-Circuit)

11.2 Event Emit Points (Notification)

11.3 The Dual Emission Pattern

Many call sites fire both a Hook and an Event for the same occurrence. For example, `Controller::redirect()` runs the `redirect` hook (short-circuit) AND emits the `redirect` event. This dual emission is the pattern across the codebase: Hooks are for modules that want to *change* the behaviour; Events are for modules that want to *observe* it.

12 The Full Rendering Pipeline Trace

This trace walks through the rendering of the home page (`common/home`) for a non-logged-in customer on a desktop browser, with `config_seo_url` on, `config_render_css_in_file` on, `config_minified_html` off, and the `choroni` theme active.

12.1 Phase 1 — Entry and Bootstrap (Steps 1–5)

1. `web/index.php` detects the store (URL path folder match), sets `STORE_ID`, `DIR_APPLICATION`, etc., includes `app/shop/config.php` and `app/shop/map.php`.
2. `app/shop/map.php` boots the registry, loads settings from `setting` table, detects language/currency, loads libraries, detects device (desktop → no template swap), loads `deps.php` for the active theme (populates `$js_assets`, `$js_header_assets`, `$jsx_assets`, `$css_assets`), filters the manifest by the current route (`common/home`) and pushes matching entries into `$javascripts`, `$header_javascripts`, `$styles`, `$scripts`.

Table 7: The 18 Hook emit points in the rendering pipeline.

Hook	Fired by	Effect of truthy return
render	Controller::render()	Bypasses the entire render; the truthy value is returned as the HTML.
fetch	Controller::fetch()	Bypasses the template execution; the truthy value is returned as the HTML.
loadWidgets	Controller::loadWidgets()	Bypasses the widget query; the truthy value is used as the widget tree.
loadAssets	Controller::loadAssets()	Bypasses the asset file discovery.
redirect	Controller::redirect()	Bypasses the header('Location:') call.
index	ControllerModuleModuleController::index()	Bypasses the widget's index method.
async	ControllerModuleModuleController::async()	Bypasses the widget's async method.
csspath	Controller::loadAssets()	Rewrites the CSS HTTP path.
cssfolder	Controller::loadAssets()	Rewrites the CSS filesystem path.
jspath	Controller::loadAssets()	Rewrites the JS HTTP path.
jsfolder	Controller::loadAssets()	Rewrites the JS filesystem path.
loadcss	ControllerCommonHeader::loadCss()	Rewrites the CSS accumulator.
loadstyles	Controller::loadAssets()	Rewrites the \$styles array.
loadjavascripts	Controller::loadAssets()	Rewrites the \$javascripts array.
processcss	startup.php	Rewrites the CSS after processing.
rowcss	Controller::loadWidgets()	Rewrites the per-row CSS.
columncss	Controller::loadWidgets()	Rewrites the per-column CSS.
widget:settings	widget controllers	Rewrites a widget's settings before rendering.
module:settings	module controllers	Rewrites a module's settings before rendering.

Table 8: The 10 Event emit points in the rendering pipeline.

Event	Fired by	Payload
addChild	Controller::addChild()	(\$child, \$params)
beforeLoad	Controller::render() child loop	(\$action, \$controller, \$params)
afterLoad	Controller::render() child loop	(\$controller)
beforeRender	Controller::render()	(\$template_path)
afterRender	Controller::render()	(\$rendered_html)
renderWidget	Controller::fetch()	(\$widget_name)
beforeLoadWidget	Controller::loadWidgets()	(\$params)
loadWidget	NecoWidget::getRows()	(\$widget)
dispatch	Front::dispatch()	(\$action)
registry:update	Registry::set()	(\$key, \$value)

3. `web/index.php` builds a `Front` controller, adds pre-actions (`common/maintenance/check`, `common/seo_url`), and calls `$controller->dispatch(new Action('common/home'), new Action('error/not_found'))`.
4. `Front::dispatch()` runs the pre-actions:
 - `common/maintenance/check`: `config_maintenance` is off, so no short-circuit.
 - `common/seo_url`: parses `?_route_` (or `?r=`) and resolves SEO URLs to `route+params` via the `url_alias` table.
5. The main action (`common/home`) is executed.

12.2 Phase 2 — Controller Setup (Steps 6–11)

6. `$this->tracker->track(0, 'home_page')` — inserts a row in the `tracker` table.
7. Cache key construction: `'html-homepage.<lang_id>...'` — the full key includes `lang_id`, `hl`, `cc`, `customer_id`, `currency`, and `store_id`.
8. `$this->cache->get($cacheId)` — cache miss (first visit).
9. `$this->document->setTitle(...)` — sets the page title from the per-language setting.
10. `$this->session->set('landing_page', 'common/home')` — stores the current route in the session (used by `NecoWidget` for `landing_page` filtering).
11. `$this->loadWidgets('featuredContent')` — loads widgets for the `featuredContent` position. This calls `NecoWidget::getRows()` which queries `property` (for rows) → `property` (for columns) → `widget` (for widgets). The widgets' children/routes are merged into `$this->children` and `$this->widget`. The rows are stored in `$this->data['rows']['featuredContent']`.

12.3 Phase 3 — Widget Loading and Child Composition (Steps 12–19)

12. `$this->loadWidgets('main')` — same for the `main` position (the center column).
13. `$this->loadWidgets('featuredFooter')` — same for the `featuredFooter` position.
14. `$this->addChild('common/column_left')` — pushes `'common/column_left'` onto `$this->children` with numeric key 0.
15. `$this->addChild('common/column_right')` — numeric key 1.
16. `$this->addChild('common/header')` — numeric key 2.
17. `$this->addChild('common/footer')` — numeric key 3.
18. `$this->cacheId = $cacheId` (only if admin is NOT logged in — enables cache write-back).
19. Template resolution: `$this->template = 'choroni/common/home.tpl'` (since `config_template` is `'choroni'` and the file exists).

12.4 Phase 4 — Render (Steps 20–28)

20. `$this->response->setOutput($this->render(true), ...)` — enters `Controller::render(true)`.
21. `runHook("render", $this, true)` — no handlers registered, returns null. Continue.
22. `$device = '.pc'` (desktop browser). `$this->cacheId .= '.pc'`.
23. `$customerLogged = 0` (not logged in). `$this->cacheId .= '0'`.

24. `$cached = $cache->get($this->cacheId, ...)` — cache miss (first visit).

25. Cache miss → asset loading:

- `loadAssets('ControllerCommonHome', 'shop')` — strips controller from classname → 'commonhome'. Looks for `web/assets/theme/choroni/css/commonhome.css` — doesn't exist, so no CSS loaded. Same for JS.
- `loadAssets('common/home', 'shop')` — strips slashes → 'commonhome' (same as above, deduplicated by `$assetLoaded`).

26. Child loop (in FIFO order):

- **Child 0:** `'common/column_left'` — `new Action('common/column_left')` → file `app/shop/controller/common/column_left.php`, class `ControllerCommonColumnLeft`, method `index`. `require_once`, `instantiate`, `call $controller->index(null)`. Inside: `loadWidgets('column_left')` → queries property + widget. If there are widgets, sets template to `choroni/common/column_left.tpl` and calls `$this->render()`. If not, does nothing (output stays null). Back in the parent: `$this->data['column_left'] = $output`.
- **Child 1:** `'common/column_right'` — same pattern, `$this->data['column_right'] = $output`.
- **Child 2:** `'common/header'` — `ControllerCommonHeader::index(null)`. Inside: IE detection, language/currency switching, token bootstrap, base URL, logo/icon, document metadata read, OpenGraph (skipped — `$params` is null), admin-mode check (no), `loadWidgets('header', 'shop', true)` → queries property + widget for the header position. `loadCss()` → reads theme table for `theme_default_id`, reads `DIR_CSS/custom-<theme_id>-choroni.css` if it exists, appends to `$this->data['css']`. `loadJs()` → concatenates `$header_javascripts` if `config_render_js_in_file`. Sets 30+ data keys. Sets `$this->id = 'header'`. Sets template to `choroni/common/header.tpl`. Calls `$this->render()`. Inside `Controller::render()` (header's own render): `loadAssets('ControllerCommonHeader', 'shop')` → looks for `commonheader.css` (doesn't exist). Child loop: no children (the header doesn't `addChild` anything). `fetch('choroni/common/header.tpl')`. Inside `fetch()`: injects magic services, `extract($this->data)`, `require header.tpl`. The template emits `<!doctype html>`, `<head>` with meta + styles + inline CSS + `header-start.tpl` (which emits the `l18n` script + polyfills), `<body>`, `#mainContainer`, `#headerContainer`, `#header` div with `widgets-rows.tpl` (which iterates `$rows['header']` and emits `{%widget_name%}` placeholders). Post-processing: strip comments, collapse whitespace. `applyFilters("render", $content)` — no filters registered. Returns the HTML. Back in the parent: `$this->data['header'] = $output`.
- **Child 3:** `'common/footer'` — `ControllerCommonFooter::index(null)`. Inside: language load, `text_powered_by`, script bucketing (4 buckets), `loadWidgets('footer')` → queries for footer widgets. `loadCss()` → route-specific CSS + per-widget CSS + minification via `NecoTool::minify()`. `loadJs()` → route-specific JS. If `config_render_js_in_file`, concatenates all `$javascripts` into the function bucket. Builds 4 `<script>` blocks. Sets `$this->id = 'footer'`. Template `choroni/common/footer.tpl`. Calls `$this->render()`. Inside `fetch()`: the footer template emits `#footerContainer`, `#footer` div with `widgets-rows.tpl`, `#copyright`, `footer-start.tpl` (which emits all `<script src>` tags + the 4-bucket `<script>` blocks + deferred `<link>` and `<style>` injection + `$().UIToTop();`), then closes `#mainContainer`, `<body>`, `<html>`. Back in the parent: `$this->data['footer'] = $output`.

- **Widget children (string keys)** — these were merged in during `loadWidgets('featuredContent')`, `loadWidgets('main')`, `loadWidgets('featuredFooter')`, and possibly `loadWidgets('header')` / `loadWidgets('footer')`. For each widget child:
 - `new Action($settings['route'])` → e.g. `'module/banner'` → `app/shop/controller/module/banner.php`, class `ControllerModuleBanner`, method `index`.
 - `require_once`, instantiate, call `$controller->index($widget)` where `$widget` is the widget row from `$this->widget[$key]`.
 - Inside `ControllerModuleModuleController::index($widget, false):` `init()` → `ControllerModuleBanner::init()` registers the `module:settings` filter. `runHook("index", $this)` — no handlers. `trigger('moduleLoad', ...)` — event. Load language `module/banner`. Unserialize `$widget['settings']`. Apply defaults. Minify `settings['style']` if set, append to `$this->css`. `loadDeps('module/banner/default')` → filters the `deps.php` manifests for matching routes. Template resolution: `choroni/module/banner.tpl` (doesn't exist) → falls back to... the banner module overrides the template in its `module:settings` filter (sets it to `choroni/banner/<jquery_plugin>.tpl`). So the template becomes e.g. `choroni/banner/nivo-slider.tpl`. Apply `module:settings` filter → `ControllerModuleBanner::init()`'s filter runs: loads the banner by `banner_id`, loads the banner items, loads per-item widgets (recursive `NecoWidget::getWidgets()` call), loads the slider JS/CSS, sets the template. `loadWidgetAssets('modulebannerdefault')` → looks for `modulebannerdefault.css/.js`. `$this->render()` → `fetch('choroni/banner/nivo-slider.tpl')` → renders the slider HTML.
- Back in the parent: `$this->data[$key . '_hook'] = $key` (e.g. `'banner_widget_1_hook' = 'banner_widget_1'`); `$this->data[$key . '_code'] = $output` (the rendered slider HTML).

27. After the child loop, the parent's own fetch: `$this->fetch('choroni/common/home.tpl')`.

- Injects magic services.
- `extract($this->data)` → `$header`, `$footer`, `$column_left`, `$column_right`, `$rows`, `$banner_widget_1_hook`, `$banner_widget_1_code`, etc. all become local variables.
- `require home.tpl`:
 - `echo $header`; — emits the header HTML.
 - `$tpl = 'choroni'`.
 - `<div id="contentContainer" class="tpl-home" nt-editable>`.
 - `include shared/widgets-featured.tpl` → sets `$position = 'featuredContent'`, includes `widgets-rows.tpl` which iterates `$rows['featuredContent']` and emits `{%<widget_name>%}` placeholders.
 - `<div id="mainContentContainer">` → `include shared/widgets-column-left.tpl` (if `$column_left`) → `echo $column_left`; (which itself was rendered by `ControllerCommonColumnLeft` and contains `{%<widget_name>%}` placeholders already substituted by that controller's own `fetch()`).
 - `include shared/widgets-column-center.tpl` → sets `$position = 'main'`, includes `widgets-rows.tpl` for the main position.
 - `include shared/widgets-column-right.tpl` (if `$column_right`).
 - `include shared/widgets-featured-footer.tpl` → `$position = 'featuredFooter'`, includes `widgets-rows.tpl`.

— `echo $footer`; — emits the footer HTML.

- First pass of widget placeholder substitution: for each widget child (string key), `str_replace('{%' . $key . '%}', $this->data[$key . '_code'], $content)`. This replaces the `{%banner_widget_1%}` tokens in the home-page HTML with the actual widget output. If a token was replaced, `unset($this->children[$key])`.
- Second pass: same loop for any remaining string-keyed children (catches tokens that appeared in the widget output itself).
- HTML post-processing: strip `/* ... */` comments, collapse doubled newlines. (`config_minified_html` is off, so no whitespace stripping.)
- `applyFilters("render", $content)` — no filters registered. Returns the final HTML.

28. `$this->output = $r` (the final HTML). Returns `$r` (because `$return` was true).

12.5 Phase 5 — Final Emission (Steps 29–33)

29. Back in `ControllerCommonHome::index()`: `$this->response->setOutput($r, $this->config->get('config_compression'))`.

30. Cache write-back: `$cache->set($this->cacheId, $r, ...)`.

31. `Front::dispatch()` loop ends (no new Action returned).

32. `web/index.php` calls `$response->output()`.

33. `Response::output()`: if `$this->level > 0`, `gzencode` the output. Send all stored headers (including `Content-Type: text/html; charset=utf-8`). `echo $output`;

12.6 Table Queries During This Render

Table 9: Database queries during a single home-page render.

Table	Read/Write	Purpose
setting	R	All config loaded at boot.
language	R	Language detection.
url_alias	R	Route resolution by <code>seo_url</code> pre-action.
tracker	W	Visit tracking.
property	R	Widget rows (<code>object_type='widget_rows'</code>) for <code>featuredContent</code> , <code>main</code> , <code>featuredFooter</code> , <code>header</code> , <code>footer</code> , <code>column_left</code> , <code>column_right</code> positions. Plus widget columns (<code>object_type='widget_cols'</code>) within each.
widget	R	Widget instances within each column.
theme	R	<code>theme_default_id</code> for the CSS file path.
content/banner	R	(If banner widget is present) banner data.
content/banner_item	R	Banner items.
content/banner_item_description	R	Banner item descriptions.

A typical home page with 5 layout positions, each with 3 rows, each with 2 columns, each with 4 widgets, generates **on the order of 165 LIKE '%...%' full-table scans** on the `property` and `widget` tables — though the widget cache (bypassed only for admins) absorbs most of these on subsequent requests.

13 The 69 Admin Modules

The admin's `app/admin/controller/module/` folder contains 69 module folders (plus 2 stray PHP files: `widget_common.php` and `widgetcontroller.php`). Each module folder typically contains:

- `install.php` — module installation hook.
- `uninstall.php` — module uninstallation hook.
- `widget.php` — the admin config controller (configures a widget instance of this module type).

Some modules also have a `plugin.php` (a plugin controller that extends functionality beyond the standard widget).

13.1 The Common Shape

`widget.php` is a script (not a class) that is require'd by `ControllerAdminModuleWidgetController`. It defines a `$return` array with the widget's configuration form fields, default settings, and the storefront route. The admin renders the form, the user fills it in, and the settings are serialised into the `widget.settings` column.

13.2 The Five Representative Modules

13.2.1 banner

A slideshow widget. Settings: `banner_id` (which banner to display), `jquery_plugin` (which slider plugin to use: `nivo-slider`, `bxslider`, etc.), `transition`, `speed`, `width`, `height`. The storefront controller (`app/shop/controller/module/banner.php`) loads the banner, loads the banner items, loads per-item widgets (recursive `NecoWidget::getWidgets()` call), loads the slider JS/CSS, and sets the template to `choroni/banner/<jquery_plugin>.tpl`.

13.2.2 category_list

A category-listing widget. Settings: `parent_id` (which category's children to list), `limit`, `view` (default, `grid`). The storefront controller loads the categories and sets the template to `choroni/module/category_list.tpl` or `category_list_grid.tpl`.

13.2.3 product_overview

A single-product-display widget. Settings: `product_id`, `show_image`, `show_price`, `show_description`, `show_add_to_cart`. The storefront controller loads the product and sets the template to `choroni/module/product_overview.tpl`.

13.2.4 shopping_cart_box

A mini-cart widget. Settings: `show_empty` (whether to show when cart is empty). The storefront controller loads the cart contents and sets the template to `choroni/module/shopping_cart_box.tpl`.

13.2.5 google_analytics

A Google Analytics tracking-code widget. Settings: `tracking_id`, `domain`. The storefront controller sets the template to `choroni/module/google_analytics.tpl`, which emits the Google Analytics JS snippet.

13.3 The Module Categories

The 69 modules fall into several categories:

Table 10: Module categories (selected).

Category	Modules
Catalog widgets	product_title, product_description, product_images, product_price, product_stock, product_attributes, product_tags, product_tabs, product_model, product_order_form, product_list, product_overview, product_filter_attributes, category_title, category_description, category_image, category_list, category_overview, manufacturer_name, manufacturer_image, manufacturer_list.
Content widgets	page_title, page_description, page_image, page_overview, post_title, post_description, post_image, post_list, post_overview, post_author, post_date_published, post_category_*, comments, facebook_comments, fblike, social, reviews, search.
Layout widgets	banner, image, lightbox, plaintext, richtext, separator, links, link_button, redirect, rooms, rooms_admin_table, web_content_crawler, plugin_template.
Store widgets	store_title, store_logo, store_phone, shopping_cart_box, shopping_cart_checkout, login_form, register_form, contact_form, invitefriends, currency_selector, language_selector.
Integration widgets	google_analytics, google_maps.

The full 69-module listing is in [Appendix A](#).

14 Closing Observations

14.1 The Presentation Layer is the Most Intricate Part of Necoyoad

The presentation layer combines seven distinct mechanisms (composite view, string-based placeholder substitution, database-backed widget tree, route-aware asset loader, `deps.php` manifest, dual Hooks/Events extension system, device-detection theme swapping) into a single coherent pipeline. No other subsystem in Necoyoad has this many interlocking parts.

14.2 The LIKE '%key=value%' Pattern is Universal

Every widget filter is a LIKE query against the serialised settings string. This works because PHP's `serialize()` produces deterministic output, but it means no index can be used (full-table scans on every widget lookup), substring false-positives are possible, and the boolean is checked by string-presence rather than by unserialising and reading the actual value. The widget cache (bypassed only for admins) absorbs most of the cost on subsequent requests.

14.3 The Composite-View Pattern Uses String Substitution, Not Tree Composition

The widget output is inserted into the parent template by `str_replace` *after* the template has been required. This is a post-render substitution, not a tree-based composition. The advantage is simplicity; the disadvantage is that a widget's output cannot itself contain a `{%...%}` token that needs to be resolved by an outer scope beyond the parent template.

14.4 The Numeric-vs-String Key Distinction is the Key Insight

The single most important thing to understand about the rendering pipeline is the numeric-vs-string key distinction in `$this->children`. Layout children (header, footer, columns) get numeric keys via `addChild()`. Widget children get string keys via `array_merge()` in `loadWidgets()`. This distinction drives everything: how the output is published into the view-model, whether it becomes `$header` or `$<name>_code`, and whether the `{%...%}` substitution applies.

14.5 The Dual Hooks/Events System is Most Active in the Presentation Layer

The presentation layer fires 18 Hooks and 10 Events at well-defined points. This is where the dual extension system is most visible: Hooks for modules that want to change the behaviour (short-circuit `render`, rewrite `fetch` output, redirect asset paths); Events for modules that want to observe it (log every dispatch, track every widget load, audit every render).

14.6 The Visual Theme Editor is Built Into the HTML

The visual theme editor is not a separate admin tool — it is built into the HTML itself, via the `nt-editable`, `data-widget`, `data-position`, `data-necotienda_module` attributes on every widget wrapper. The editor's JS scans the DOM for these attributes and attaches click/drag handlers. The `/**key**/ ... /**/key**/` CSS markers let the editor find and replace a specific widget's CSS block. The `theme_style` table stores CSS-rule overrides at the selector/property/-value granularity. The generated CSS file is loaded by the header controller on every page.

14.7 The Theme Inheritance Mechanism is a Single Line of PHP

The `$tpl = is_dir(...) ? $config_template : "choroni";` line at the top of every template is the entire theme inheritance mechanism. A child theme can override any template while still inheriting the `shared/` fragments from `choroni`. This is one of the simplest and most effective theme-inheritance mechanisms in any PHP platform.

14.8 The Asset Loading is Convention-Based

The route `?r=store/product` is mapped by string convention to `storeproduct.css` and `storeproduct.js`. There is no explicit asset registration; the loader just looks for files named after the route. The `deps.php` manifest adds route-to-asset mappings for assets that don't follow the convention (e.g. a CSS file that applies to multiple routes).

14.9 The Cache is Bypassed for Admins

Every cache lookup in the rendering pipeline (`Controller::render()`, `NecoWidget::getRows()`, `NecoWidget::getWidgets()`) is bypassed if an admin is logged in. This ensures admins see changes immediately, at the cost of database load. The trade-off is deliberate: the admin experience is prioritised over admin-side performance.

14.10 Final Note

v3 confirms that the Necoyoad presentation layer is a characterful, hand-built system that combines OpenCart-derived conventions (the composite-view pattern, the `Controller::render()/fetch()` split, the `.tpl` file convention) with a proprietary widget pipeline (the `property-EAV` widget tree, the `LIKE '%key=value%'` query pattern, the `{%widget_name%}` placeholder substitution, the `deps.php` manifest, the visual theme editor). The result is a presentation layer that is flexible enough to support a visual drag-and-drop editor while remaining a straightforward PHP-template system at its core.

A The 69 Admin Module Folders

Table 11: The 69 admin module folders in `app/admin/controller/module/`.

Module	Purpose
<code>banner</code>	Slideshow widget (nivo-slider, bxslider, etc.).
<code>category_description</code>	Display a category's description.
<code>category_image</code>	Display a category's image.
<code>category_list</code>	List categories (default or grid view).
<code>category_overview</code>	Display a category overview.
<code>category_title</code>	Display a category's title.
<code>comments</code>	Display comments on a post/product.
<code>contact_form</code>	Display a contact form.
<code>currency_selector</code>	Display a currency selector dropdown.
<code>facebook_comments</code>	Display Facebook comments plugin.
<code>fblike</code>	Display a Facebook Like button.
<code>google_analytics</code>	Emit Google Analytics tracking code.
<code>google_maps</code>	Display a Google Maps embed.
<code>image</code>	Display a single image.
<code>invitefriends</code>	Display an invite-friends form.
<code>language_selector</code>	Display a language selector dropdown.
<code>lightbox</code>	Display a lightbox image.
<code>link_button</code>	Display a link button.
<code>links</code>	Display a list of links.
<code>login_form</code>	Display a customer login form.
<code>manufacturer_image</code>	Display a manufacturer's image.
<code>manufacturer_list</code>	List manufacturers.
<code>manufacturer_name</code>	Display a manufacturer's name.
<code>page_description</code>	Display a page's description.
<code>page_image</code>	Display a page's image.
<code>page_overview</code>	Display a page overview.
<code>page_title</code>	Display a page's title.
<code>plaintext</code>	Display plain text.
<code>plugin_template</code>	Scaffolding boilerplate for new modules.
<code>post_author</code>	Display a post's author.
<code>post_category_description</code>	Display a post category's description.
<code>post_category_image</code>	Display a post category's image.
<code>post_category_list</code>	List post categories.
<code>post_category_overview</code>	Display a post category overview.
<code>post_category_title</code>	Display a post category's title.
<code>post_date_published</code>	Display a post's publication date.
<code>post_description</code>	Display a post's description.
<code>post_image</code>	Display a post's image.
<code>post_list</code>	List posts.
<code>post_overview</code>	Display a post overview.
<code>post_title</code>	Display a post's title.
<code>product_attributes</code>	Display a product's attributes.
<code>product_description</code>	Display a product's description.
<code>product_filter_attributes</code>	Display product filter attributes.
<code>product_images</code>	Display a product's images.
<code>product_list</code>	List products.
<code>product_model</code>	Display a product's model.

Module	Purpose
product_order_form	Display a product order form.
product_overview	Display a product overview.
product_price	Display a product's price.
product_stock	Display a product's stock status.
product_tabs	Display product tabs.
product_tags	Display a product's tags.
product_title	Display a product's title.
redirect	Redirect to a URL.
register_form	Display a customer registration form.
reviews	Display reviews on a product/post.
richtext	Display rich-text (HTML) content.
rooms	Display rooms (custom module).
rooms_admin_table	Admin table for rooms.
search	Display a search box.
separator	Display a visual separator.
shopping_cart_box	Display a mini-cart.
shopping_cart_checkout	Display a checkout mini-cart.
social	Display social-media links.
store_logo	Display the store's logo.
store_phone	Display the store's phone number.
store_title	Display the store's title.
web_content_crawler	Web content crawler widget.

B Theme Folder Tree

```
app/shop/view/theme/choroni/
|-- account/          (account page templates)
|-- banner/           (slideshow templates: nivo-slider.tpl, bxslider.tpl, ...)
|-- checkout/         (checkout flow templates)
|-- common/           (layout templates: header.tpl, footer.tpl,
|                       home.tpl, column_left.tpl, column_right.tpl,
|                       maintenance.tpl, success.tpl, admin/)
|-- content/          (CMS content templates: post, page, category)
|-- error/            (error page templates: not_found.tpl, etc.)
|-- localisation/     (localisation templates)
|-- module/           (69 widget templates, one per module type)
|-- page/             (static page templates)
|-- payment/          (payment method templates: PayU, PayPal, etc.)
|-- shared/           (shared fragments included by every template)
|   |-- fields/       (form field partials)
|   |   |-- payment/  (payment-specific field partials)
|   |   '-- shipping/ (shipping-specific field partials)
|   |-- fragment/     (header-start.tpl, footer-start.tpl)
|   '-- product/      (product-related fragments)
|-- store/            (storefront catalog templates: product.tpl,
|                       category.tpl, manufacturer.tpl, etc.)
'-- common/admin/     (admin-mode templates shown when admin is logged in)
```

B.1 Selected File Counts

Table 12: Selected theme folder file counts.

Folder	.tpl files	Role
common/	7	Layout templates (header, footer, home, columns, maintenance, success).
shared/	12	Fragments included by every template (widgets-rows, widgets-column-*, fragment/*, widget-head, etc.).
module/	69	One template per widget type (plus variants like <code>_default.tpl</code> , <code>_grid.tpl</code>).
store/	14	Catalog templates (product, category, manufacturer, special, search, review, etc.).
account/	22	Account page templates (login, register, address, order, payment, balance, message, etc.).
checkout/	8	Checkout flow templates (cart, confirm, success, etc.).
content/	8	CMS templates (post, page, category).
banner/	22	Slideshow templates (one per jQuery slider plugin variant).
payment/	4	Payment method templates (PayU, PayPal, bank transfer, etc.).
page/	4	Static page templates.
error/	2	Error page templates.
localisation/	2	Localisation templates.

C Glossary

Table 13: Glossary of presentation-layer terms.

Term	Meaning
Composite view	A pattern where a page is assembled from a parent template plus layout children plus widget children.
Layout child	A child controller (header, footer, column_left, column_right) added via <code>addChild()</code> with a numeric key.
Widget child	A child controller (a positioned, configurable content block) merged in via <code>loadWidgets()</code> with a string key.
<code>{%widget_name%}</code>	A placeholder token in a template, replaced by the widget's output via <code>str_replace</code> after the template is required.
NecoWidget	The widget data-access object (<code>system/helper/widgets.php</code>). Queries the <code>property</code> EAV table for widget rows, columns, and instances.
LIKE '%key=value%'	The query pattern used to filter widgets by their serialised settings. Works because PHP's <code>serialize()</code> is deterministic, but forces full-table scans.
<code>deps.php</code>	The asset manifest file (one per theme, one for CSS, one for JS). Declares which CSS/JS files apply to which routes.
<code>config_template</code>	The setting that names the active theme folder. Swapped at boot for mobile/tablet/Facebook devices.
<code>config_render_css_in_file</code>	Per-store setting. When on, CSS is inlined; when off, via <code><link></code> tags.
<code>nt-editable</code>	HTML attribute on every widget wrapper. The visual theme editor scans the DOM for this attribute.
<code>theme_style</code>	Table storing CSS-rule overrides at the selector/property/value granularity.
<code>\$tpl</code>	The theme-inheritance variable. If the active theme has a <code>/shared</code> folder, use it; otherwise fall back to <code>'choroni'</code> .
Magic services	The five services injected into every template by <code>Controller::fetch(): \$Config, \$Language, \$I, \$Request, \$Url, \$Image</code> .